

Pioneer 3-DX en Coppeliasim y Lua

Celda de almacén con R1, Bill, herramientas T1/T2, evitación reactiva, control y SLAM.

Fase 1

Fase 2

Control

SLAM

Autor: Jorge Luis Mayorga Taborda
Profesor: Jose I. Iñiguez
26 de mayo de 2026

Fondo: celda automatizada real

Ruta del trabajo

Escena

Control

SLAM

Fase 2

Cierre

Recorre escena, control, seguimiento, SLAM, métricas y límites.

01**Escena y requisito**

mannequin/Bill, celda de almacén, R1, T1/T2 y métricas

02**Arquitectura y código**

funciones Lua, FSM, sensores, planner y logger

03**Control de seguimiento**

P, PI, PID, LQR y NMPC con coste y pesos definidos

04**SLAM y Fase 2**

Kalman, grids log-odds, scan-to-map y submapas simplificados

05**Resultados**

tablas, límites, elección técnica y mejoras

Orden

Cada bloque liga requisito, código, métrica y límite.

Mapa operativo de la celda

Escena

Control

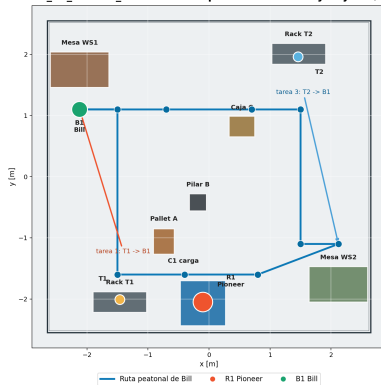
SLAM

Fase 2

Cierre

La escena se organiza alrededor del rack T1/T2, mesas WS1/WS2, Bill, R1 y la estación de carga.

Sim_T2_Phase1_Basic: vista superior de la celda y flujo B1/R1



Secuencia

- R1 parte de carga y toma T1 en el rack.
- Bill trabaja en WS1 y después se atiende WS2.
- La ruta evita pallet, pilar y caja durante el ciclo.
- Los estados exportados conectan misión, control y SLAM.

T1/T2 **R1/B1**
cola de herramientas robot y persona

Requisitos y objetivos

Escena

Control

SLAM

Fase 2

Cierre

Se modela una celda tipo almacén donde un Pioneer P3DX debe seguir a una persona, entregar herramientas, evitar obstáculos y construir una representación local del entorno.

Nomenclatura: el *mannequin* de la especificación se implementa en la escena como Bill/B1; el requisito de seguimiento es el mismo.

Requisito

Coordinar seguimiento, navegación diferencial, seguridad local y tareas T1/T2 en una escena con persona móvil y obstáculos.

Implementación

Control reactivo con campos potenciales, FSM de misión, estimación Kalman y comparación de controladores.

Mapa

Mapeo con Kalman-landmark, grids log-odds, scan-to-map, submapas simplificados y Fase 2 con mapa desconocido.

Objetivos evaluados

- Regular distancia y orientación respecto a Bill mediante un punto de referencia p^* .
- Planificar entregas T1/T2 con una máquina de estados reproducible.
- Evitar obstáculos con sensores de proximidad, reducción de velocidad y recuperación.
- Comparar controladores y métodos SLAM con métricas comunes.

Arquitectura integrada

Escena

Control

SLAM

Fase 2

Cierre

El sistema se evalúa como un ciclo cerrado: los sensores alimentan estimación y mapa; el planificador local elige un objetivo intermedio; el controlador diferencial genera las velocidades de rueda y el logger exporta las señales.

Sensores

16 ultrasonidos, rayos visuales y distancia mínima.

Estimador

odometría, corrección y error de pose.

Mapa

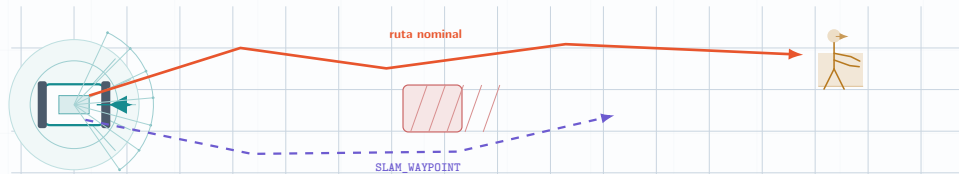
landmarks, celdas, submapas y cierres.

Planner

ruta directa o SLAM_WAYPOINT.

Control

P/PID/PID, LQR o NMPC.

`readObstacleField`

riesgo, giro y distancia mínima

`registerSlamDetection`

impacto sensor a mundo y mapa

`plannedWaypointTo`

ruta directa o punto intermedio

`publishTelemetry`

CSV/JSON de estado y mapa

Almacén: R1, Bill y herramientas

Escena

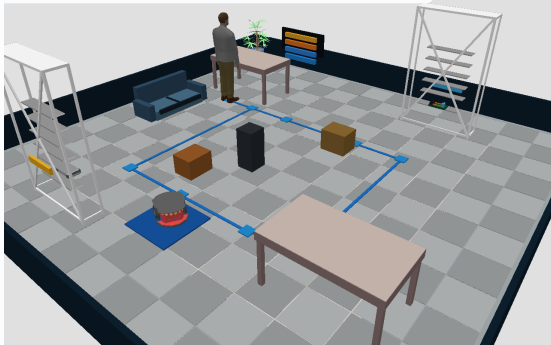
Control

SLAM

Fase 2

Cierre

Escena principal: R1, Bill, rack T1/T2, mesas WS1/WS2 y estación de carga



Operación en escena

- La misión completa se ejecuta en una celda de almacén.
- Al entregar, la pieza queda *sobre* la mesa ($z \approx 0,73$ m) y Bill mueve las manos sobre ella; al devolverla, desaparece de la mesa.
- La evitación local y el mapa SLAM se actualizan durante la misma ejecución.
- El cierre se mide con estado, batería, riesgo, pose y mapa exportados.

4/4

tareas completadas

0,034 m

error final de pose

Escenas de simulación

Escena

Control

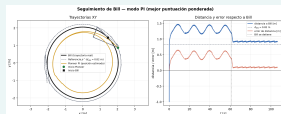
SLAM

Fase 2

Cierre

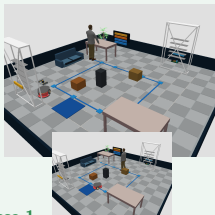
Tres ejecuciones registradas

Las capturas cubren tres estados del sistema: seguimiento, entrega con evitación y mapa desconocido.



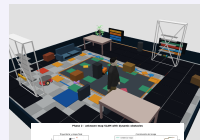
Follower

Bill define el objetivo móvil y se comparan P, PI, PID, LQR y NMPC.



Fase 1

Entrega T1/T2, Bill trabajando y evitación durante la misma misión.



Fase 2

Mapa desconocido, obstáculo dinámico y reconstrucción exportada.

Capturas de ejecución

Escena

Control

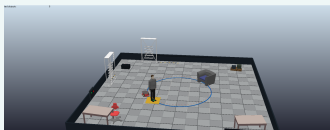
SLAM

Fase 2

Cierre

Ejecuciones registradas en CoppeliaSim

Follower registra seguimiento circular; Fase 2 registra zona desconocida, ruta de navegación, elementos de la celda y sensores de proximidad.



Follower: trayectoria circular de Bill



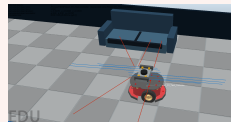
Follower: R1 regula distancia sin cortar la marcha



Fase 2: obstáculo no modelado



Fase 2: vista general de la celda



Fase 2: sensores frente al sofá

Video de ejecución I

Escena

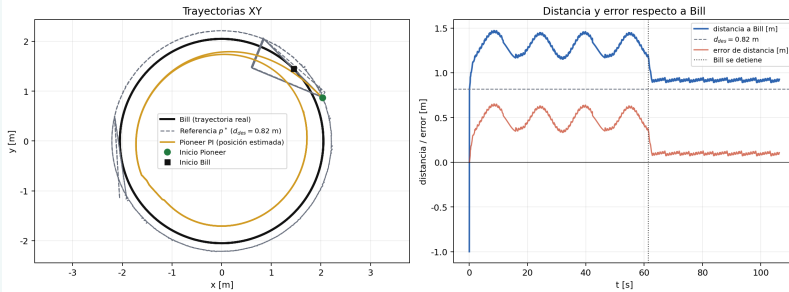
Control

SLAM

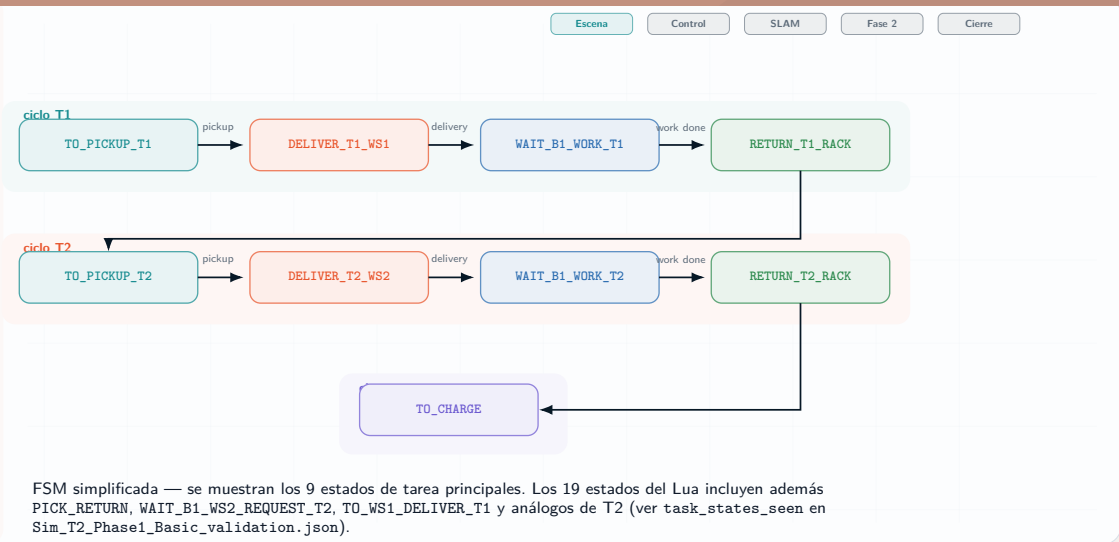
Fase 2

Cierre

Seguimiento de Bill — modo PI (mejor puntuación ponderada)

Seguimiento de Bill, entrega T1/T2 y retorno a carga · [figures/Video_Follower.mp4](#)

Máquina de estados de R1



Funciones Lua integradas

Escena

Control

SLAM

Fase 2

Cierre

Funciones usadas: lectura de sensores, registro SLAM, waypoint local y entrega a Bill.

`readObstacleField()`

Lee 16 sensores, calcula risk, minObstacle, giro repulsivo y reducción de velocidad.

`registerSlamDetection(sensor,d,pt)`

Transforma el impacto al marco global y actualiza landmark, celda log-odds o submapa.

`plannedWaypointTo(goal)`

Comprueba si la ruta directa está bloqueada y devuelve SLAM_WAYPOINT o el objetivo final.

`deliverToolToBill(tool,ws,next)`

Ejecuta la transición de FSM: aproxima R1, entrega T1/T2, espera a Bill y registra el estado.

Seguimiento de Bill

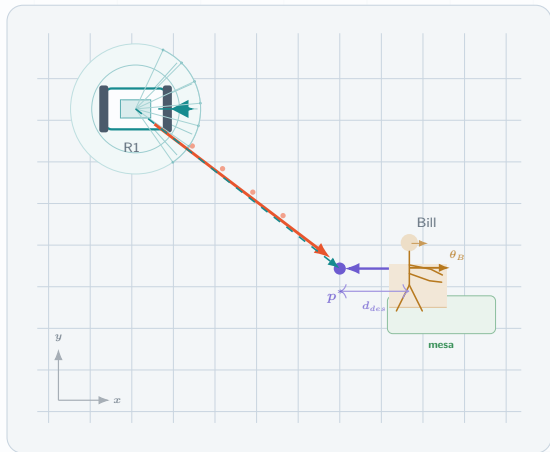
Escena

Control

SLAM

Fase 2

Cierre



$$p^* = p_B - d_{des} [\cos \theta_B, \sin \theta_B]^T$$

Control sobre el error

Los cinco controladores reciben la misma geometría. Cambia la ley de mando, no la trayectoria de Bill ni el logger.

P / PI / PID

LQR

NMPC

0,82 m

distancia deseada usada en seguimiento

mismo CSV

comparación bajo la misma trayectoria

Cinemática diferencial

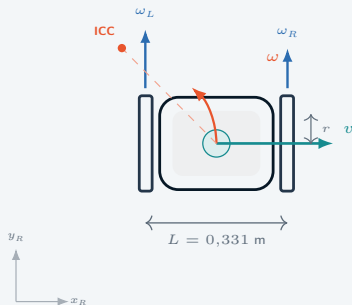
Escena

Control

SLAM

Fase 2

Cierre



$$\omega_L = \frac{v - \frac{L}{2}\omega}{r}$$

$$\omega_R = \frac{v + \frac{L}{2}\omega}{r}$$

Parámetros usados

0,0975 m

radio de rueda

0,62 m/s

$v_{\max}$ controlador misión

1,42 rad/s

$\omega_{\max}$ controlador misión

Control de seguimiento: P, PI y PID

Escena

Control

SLAM

Fase 2

Cierre

Los tres modos usan el mismo error de distancia y el mismo error de rumbo. Cambia la memoria del controlador: presente, acumulado y derivada.

P

$$u_P = K_p e_d$$

Responde al error instantáneo. Es estable y simple, pero deja offset.

PI

$$u_{PI} = K_p e_d + K_i I_d$$

$$I_d = \sum e_d \Delta t$$

Añade memoria integral para corregir error sostenido durante el seguimiento.

PID

$$u_{PID} = K_p e_d + K_i I_d + K_d \dot{e}_d$$

$$I_d = \sum e_d \Delta t$$

El término derivativo amortigua cambios bruscos, pero sube el esfuerzo.

Errores comunes a los tres modos

Error de distancia

$$e_d = \|p_B - p_R\| - d_{des}$$

Error de rumbo

$$\phi_B = \text{atan2}(y_B - y_R, x_B - x_R)$$

$$e_\psi = \text{wrap}(\phi_B - \theta_R)$$

La salida se convierte en v y ω , y después en ω_L, ω_R . Todos los modos pasan por saturaciones de velocidad y aceleración antes de mandar las ruedas.

Control de seguimiento: LQR discreto

Escena

Control

SLAM

Fase 2

Cierre

LQR regula el error local alrededor de p^* . La matriz K se obtiene resolviendo la ecuación de Riccati discreta en la propia escena (en Lua, solveDiscreteLqr). El solver se verifica fuera de línea contra SciPy.

Estado y control

$$x_L = [e_{long}, e_{lat}, e_{yaw}]^T$$

$$u = [\Delta v, \Delta \omega]^T$$

e_{long} y e_{lat} : proyección de $p^* - p_R$ en el marco de R1; $e_{yaw} = e_{\psi}$.

Modelo discreto

$$A_d = I + \Delta t A$$

$$B_d = \Delta t B$$

Ley aplicada

$$u = -Kx_L$$

Luego se saturan v y ω .

Función cuadrática

$$J = \sum (x_k^T Q x_k + u_k^T R u_k)$$

Q diag

35, 80 y 18 para error longitudinal, lateral y yaw

R diag

6 y 3 para variación de avance y giro

$$\Delta t = 0,05 \text{ s}$$

misma escala de simulación

Control de seguimiento: NMPC

Escena

Control

SLAM

Fase 2

Cerrar

NMPC simula el modelo no lineal hacia adelante y elige una secuencia corta de comandos. En esta implementación se resuelve por disparo directo con búsqueda local.

Modelo predictivo

$$x_{k+1} = x_k + v_k \cos \theta_k \Delta t$$

$$y_{k+1} = y_k + v_k \sin \theta_k \Delta t$$

$$\theta_{k+1} = \theta_k + \omega_k \Delta t$$

Coste resumido

$$J = \sum_k (c_e + c_u + c_s) + q_f \|p_N^* - p_N\|^2$$

$$c_e = q_d e_d^2 + q_t \|p_k^* - p_k\|^2 + q_\psi e_\psi^2$$

$$c_u = r_v v_k^2 + r_\omega \omega_k^2 + r_{whl} (\omega_L^2 + \omega_R^2)$$

$$c_s = s_v \Delta v_k^2 + s_\omega \Delta \omega_k^2$$

Horizonte

$N = 10$, $\Delta t = 0,22$ s; patrón Hooke-Jeeves con pasos decrecientes.

Límites

$v \in [-v_{rev}, v_{max}]$, $\omega \in [-\omega_{max}, \omega_{max}]$.

Pesos usados

$q_d = 48$, $q_t = 7$, $q_\psi = 1, 2$, $r_v = 1, 4$, $r_\omega = 1, 8$, $r_{whl} = 0, 020$, $s_v = 65$, $s_\omega = 18$, $q_f = 6$.

La versión Lua conserva modelo no lineal, horizonte, restricciones y coste predictivo. La optimización se resuelve con una búsqueda directa pequeña para ejecutarse dentro de CoppeliaSim.

Resultados de seguimiento

Escena

Control

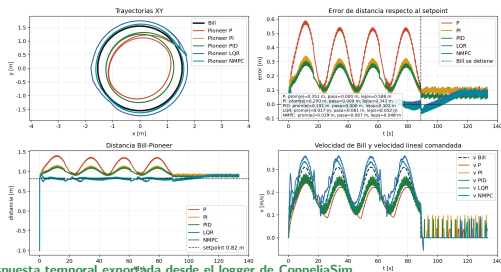
SLAM

Fase 2

Cierre

Misma misión y mismo logger: pondera precisión, ruta, batería y riesgo.

Actividad 2.1 - Pioneer siguiendo a Bill en círculo: P, PI, PID, LQR y NMPC



Respuesta temporal exportada desde el logger de CoppeliaSim

PI

mejor balance operativo

NMPC

menor ruta y batería

LQR

menor riesgo máximo

PID

menor error final

Métrica

Puntaje = suma min-max normalizada, menor es mejor: 0,28 movimiento + 0,18 batería + 0,20 variación + 0,14 riesgo + 0,12 error de pose + 0,08 duración. Riesgo = $\max_t \text{obstacle_risk}(t)$.

Coste de ruedas, energía y suavidad

Escena

Control

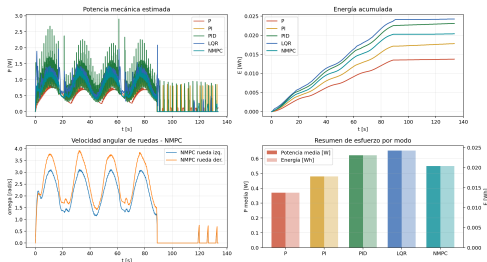
SLAM

Fase 2

Cierre

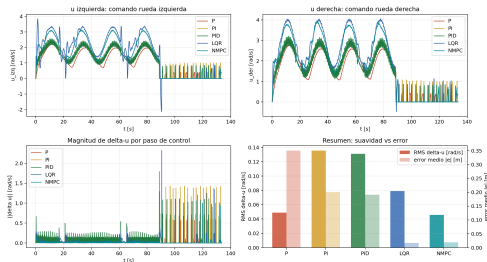
El error final no basta: las ruedas muestran energía, suavidad y picos de mando.

Actividad 2.1 - Datos de ruedas y potencia estimada



Potencia y energía acumulada

Actividad 2.1 - Suavidad de comandos de rueda u y δu



Suavidad del comando

Evación de obstáculos

Escena

Control

SLAM

Fase 2

Cierre

02

Evación reactiva de obstáculos

El robot detecta obstáculos con 16 sensores de proximidad y aplica campos potenciales para desviar la trayectoria en tiempo real durante la misión.

campos potenciales

16 ultrasonidos

recuperación activa

Video: evasión y misión completa

Escena

Control

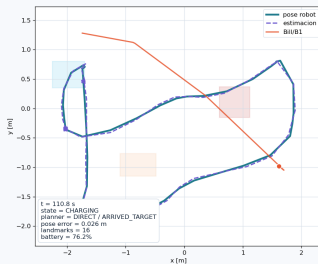
SLAM

Fase 2

Cierre

Replay visual desde logs de CoppeliaSim

Trayectoria real, estimación SLAM, objetivo humanoide y telemetría de misión.



Seguimiento de Bill, entrega T1/T2, evitación reactiva y SLAM · figures/Video_Task_And_Avoid_Obstacle.mp4

Campos potenciales y anticolisión

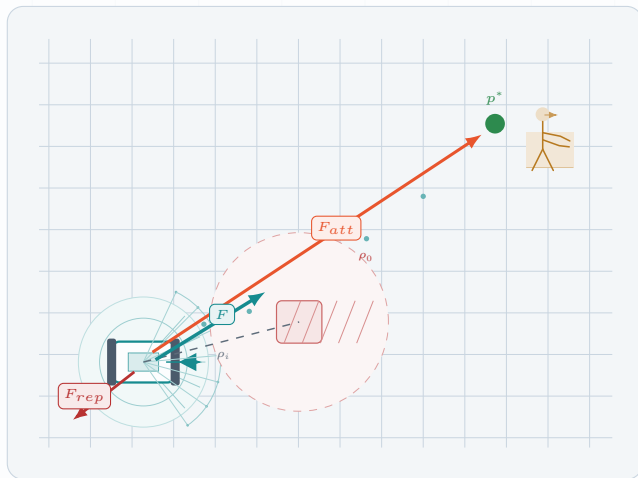
Escena

Control

SLAM

Fase 2

Cierre



Ley usada

$$\vec{F}_i = \vec{F}_{att} + \sum_i \vec{F}_{rep,i}$$

El rumbo se recalcula en cada ciclo con objetivo, sensores, límites de velocidad y modo de recuperación.

16

ultrasonidos del Pioneer

0,62 m

rango de influencia reactiva

0,10 m

parada crítica frontal

Sensores: LiDAR y Pioneer

Escena

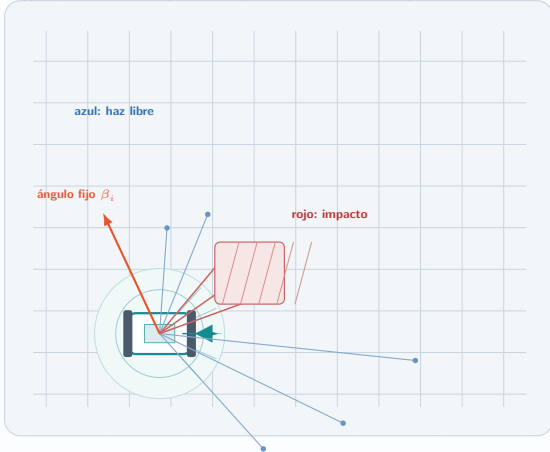
Control

SLAM

Fase 2

Cierre

La escena no añade un LiDAR físico 2D. Usa los 16 sensores de proximidad (sonar) del Pioneer como rayos discretos: cada lectura entrega ángulo, distancia e impacto. En pantalla el haz se dibuja **azul** si no detecta nada y **rojo** si toca un obstáculo.



Cómo se usa

$$z_i = (r_i, \beta_i)$$

$$p_i^W = [\hat{x}, \hat{y}]^T + R(\hat{\theta})[r_i \cos \beta_i, r_i \sin \beta_i]^T$$

- LiDAR real: muchos rayos láser, contornos densos y mejor resolución angular.
- Pioneer simulado: 16 sensores; suficientes para evitar obstáculos y mapear localmente.
- GMapping, Hector y Cartographer reciben estos rayos discretos, no un paquete ROS completo.
- La baja densidad explica por qué los mapas son aproximados y por qué Kalman funciona bien en una celda pequeña.

16 sensores

pose 2D

Planner local con SLAM_WAYPOINT

Escena

Control

SLAM

Fase 2

Cierre

El ciclo estima pose, transforma sensores, actualiza el mapa y devuelve un waypoint al controlador diferencial.



Kalman-landmark

$$P_k^- = G_k P_{k-1}^+ G_k^T + Q_k$$

$$K_k = P_k^- H_k^T (H_k P_k^- H_k^T + R_k)^{-1}$$

$$\hat{x}_k^+ = \hat{x}_k^- + K_k (z_k - H_k \hat{x}_k^-)$$

En la escena corrige pose con medición ruidosa y asocia detecciones por cercanía. Si no hay asociación, crea un landmark no asociado.

pose 2D

$[x, y, \theta]$, sin roll/pitch

8-10

landmarks finales (Kalman)

WAYPOINT

SLAM_WAYPOINT usa obstáculos mapeados

SLAM Kalman-landmark

Escena

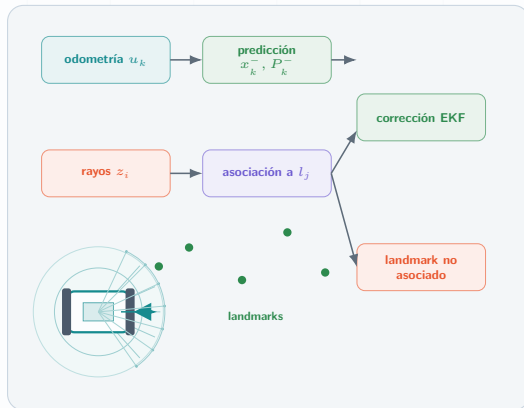
Control

SLAM

Fase 2

Cierre

El mapa se representa como puntos observados; la incertidumbre se mantiene en la pose del robot.



Modelo usado

$$\hat{x}_k^- = f(\hat{x}_{k-1}, u_k)$$

$$P_k^- = F_k P_{k-1} F_k^T + Q_k$$

$$K_k = P_k^- H_k^T (H_k P_k^- H_k^T + R_k)^{-1}$$

Cada impacto de sensor se transforma al marco del almacén. Si cae cerca de un landmark existente, corrige la pose; si no, se registra como punto de mapa.

0,017 m

menor RMSE Fase 1
(GMapping)

8–10

landmarks finales (Kalman)

límite

no modela paredes como
áreas ocupadas

Grid log-odds inspirada en GMapping

Escena

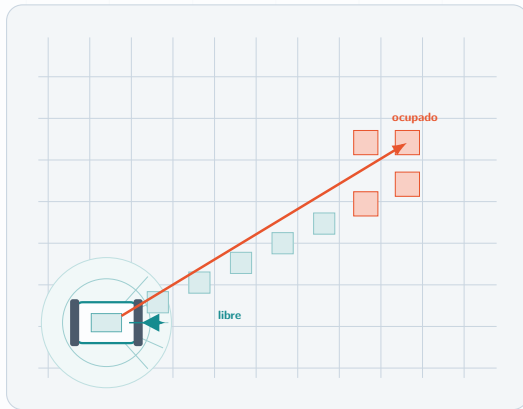
Control

SLAM

Fase 2

Cierre

El mapa pasa de puntos aislados a celdas libres u ocupadas, útiles para evitar bloqueos locales.



Actualización

$$L_t(c) = \text{clip}(L_{t-1}(c) + \ell(c|z_t, \hat{x}_t))$$

$\ell(c) = \ell_{free}$ en celdas atravesadas

$\ell(c) = \ell_{occ}$ en el impacto

La versión Lua implementa una grid de ocupación por log-odds con la pose estimada. No es GMapping completo: no hay filtro de partículas ni árbol de trayectorias.

94

celdas ocupadas Fase 1

36

rasgos Fase 2

límite

la calidad depende de la pose usada para trazar rayos

Scan-to-map inspirado en Hector

Escena

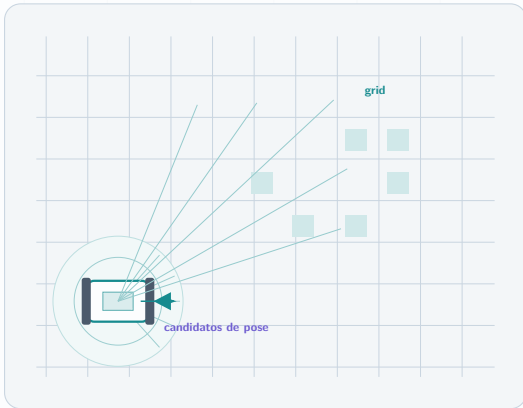
Control

SLAM

Fase 2

Cierre

La aproximación scan-to-map corrige la pose buscando el desplazamiento que mejor encaja el escaneo actual con la grid.



Regla de ajuste

$$\xi^* = \arg \max_{\xi} \frac{1}{N} \sum_i M(T_{\xi} p_i)$$
$$x_k \leftarrow x_k^- + \alpha(\xi^* - x_k^-)$$

Se evalúan pequeñas correcciones alrededor de la pose predicha. Es una aproximación Lua scan-to-map, no la pila Hector completa con optimización multi-resolución.

15,233 m

ruta más corta Fase 1

0,333

precisión final de rasgos Fase 2

límite

riesgo de óptimos locales con poca geometría

Submapas inspirados en Cartographer

Escena

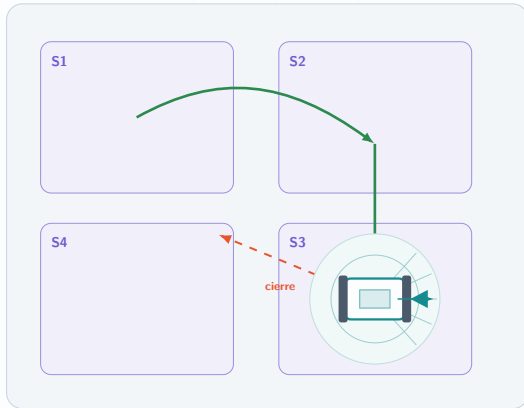
Control

SLAM

Fase 2

Cierre

El mapa se organiza en submapas; los retornos a zonas conocidas generan cierres de ciclo simplificados.



Lógica

scan → submapa activo

distancia/tiempo → submapa adicional

score alto con submapa antiguo → cierre

La versión Lua conserva la idea de submapas y restricciones locales, pero no resuelve una optimización global de grafo ni loop closure completo como Cartographer.

13

submapas finales

4

cierres de ciclo

89,5 %actividad de mapa
(submapas)

Qué SLAM usar y por qué

Escena

Control

SLAM

Fase 2

Cierre

Kalman

localización

pose estable y pocos rasgos

no dibuja paredes

Log-odds

ocupación de celda

costmap local y obstáculos

sin partículas RBPF

Hector

ajuste de scan

corregir pose con geometría

oscila con pocos rayos

Submapas

revisita y cierre

zonas revisitadas y cierres

más coste y filtros

Resultados SLAM Fase 1

Escena

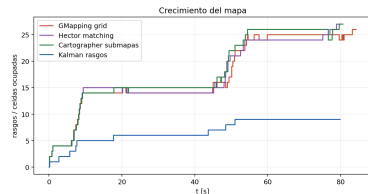
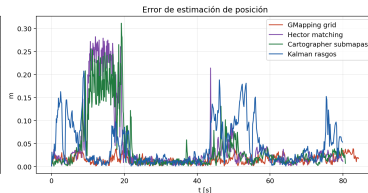
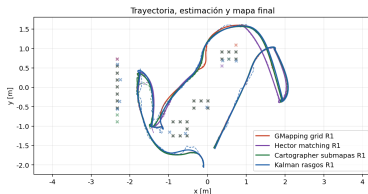
Control

SLAM

Fase 2

Cierre

Trayectoria, error y mapa exportados desde Fase 1 — cuatro estimadores



Timeline de misión

Escena

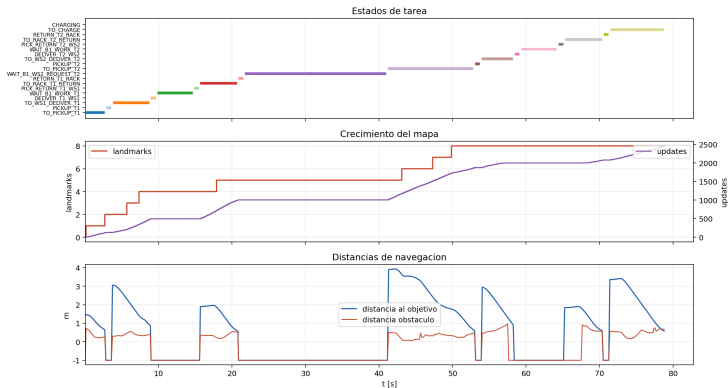
Control

SLAM

Fase 2

Cierre

Estados de tarea, movimiento, planner, Bill y batería en la misma escala temporal



Fase 2: mapa desconocido

Escena

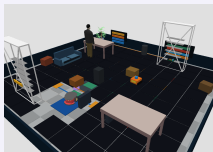
Control

SLAM

Fase 2

Cierre

Fase 2 arranca con parte de la celda desconocida y obliga a descubrir obstáculos con rayos de proximidad mientras el robot mantiene la misión T1/T2. La zona sin mapear se dibuja como una **niebla oscura** (rejilla de 100 baldosas) que R1 va despejando a su paso conforme la explora.



niebla de exploración: celda por descubrir



replanificación ante obstáculo dinámico

0,017 m

GMapping: menor RMSE de pose

91,3 %

submapas: actividad de mapa

14–20

replanificaciones según modo

0–1

recuperaciones del planner

Fase 2: exploración y SLAM que converge

Escena

Control

SLAM

Fase 2

Cierre

La misión arranca con una pasada exploratoria que mapea la celda antes de operar y luego repite el ciclo de trabajo. El aprendizaje no se ve en la velocidad (la domina el robot móvil), sino en la calidad del SLAM: el mapa converge y la localización se afina vuelta a vuelta.

Explora primero

recorre *frontiers* de los pasillos centrales y mapea con el sonar antes de tocar piezas (EXPLORE_FRONTIERS)

Repite el ciclo

tras explorar ejecuta tres veces el ciclo T1/T2; el mapa SLAM y la pose se conservan entre vueltas

Mapa converge

los *landmarks* nuevos por fase caen de ≈ 4 a ≈ 1 : deja de descubrir estructura

Localización se afina

el error de pose medio baja de 0,086 m sin mapa a $\approx 0,03$ m con el mapa construido

Calidad del SLAM por fase (media de 3 ejecuciones)

Explorar

landmarks: 4 (+4 nuevas)
error pose: 0,086 m

Vuelta 1

landmarks: 9 (+4 nuevas)
error pose: 0,033 m

Vuelta 2

landmarks: 10 (+1 nuevas)
error pose: 0,042 m

Vuelta 3

landmarks: 11 (+1 nuevas)
error pose: 0,031 m

3

vueltas completas sobre la celda

4 → 1

landmarks nuevos: mapa converge

2,6×

mejor localización con mapa

0

recuperaciones en 3 ejecuciones

Recuperación local en Fase 2

Escena

Control

SLAM

Fase 2

Cierre

Las variantes de grid se bloquean cuando el mapa trata como obstáculo cualquier celda cercana al segmento hacia el objetivo. El filtro separa bloqueo real, obstáculo pegado al robot y obstáculo pegado al punto de recogida.

Persistencia

la celda se marca como bloqueante solo tras detecciones repetidas en el log-odds

Distancia local

se ignoran celdas dentro del radio de R1 o pegadas al objetivo para no cerrar el punto final

Área útil

el coste del waypoint penaliza puntos fuera del pasillo operativo del almacén

Recuperación

si se estanca, retrocede en arco (RECOVER_BACK); tras varios intentos gira 180° (TURN_AROUND) para salir por otro lado; geo-valla X/Y/Z evita escape y caída

mapa crudo

filtros

waypoint

control reactivo

objetivo

4/4

métodos completan Fase 2

1

recuperación observada en submapas

17

mínimo de replanificaciones, Kalman

25

máximo de replanificaciones, submapas

Comparación Fase 2: trayectorias

Escena

Control

SLAM

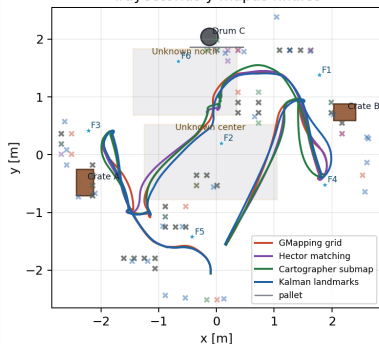
Fase 2

Cierre

La comparación se hace con la misma misión T1/T2 y los mismos obstáculos dinámicos.

Trayectorias finales y rasgos detectados

Trayectorias y mapas finales



Comparación

- Kalman: trayectoria más estable en error de pose.
- Scan-to-map y log-odds: ruta cercana, con más dispersión local.
- Submapas: cierres simplificados y más estructura.
- El robot móvil dinámico modifica el recorrido sin cambiar la misión.

4

modos SLAM

14-20

replanificaciones

4

cruces dinámicos

0-1

recuperaciones

Comparación Fase 2: cobertura temporal

Escena

Control

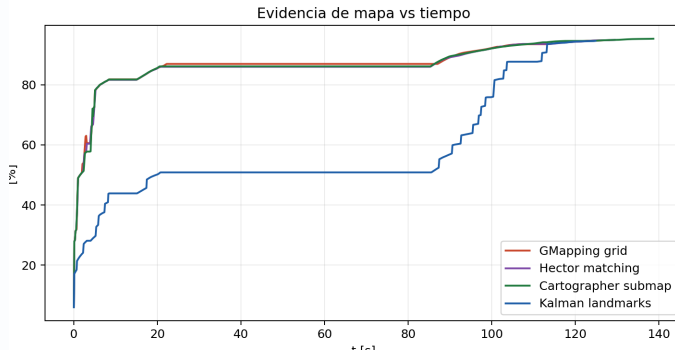
SLAM

Fase 2

Cierre

La cobertura de mapa se registra durante la ejecución y compara la velocidad de descubrimiento.

Curva de cobertura de mapa



Comportamiento

Las grids suben rápido al inicio. Kalman descubre rasgos más despacio, pero termina cerca del mismo rango de cobertura.

5,1 s

75 % de cobertura en grids

98,55 s

75 % de cobertura en Kalman

Comparación Fase 2: cobertura final

Escena

Control

SLAM

Fase 2

Cierre

La actividad de mapa de los métodos de grid ronda 88–92 %; Kalman-landmark queda en 51 % por fusionar pocas landmarks discretas.



Comparación Submapas alcanza la mayor cobertura final, pero la diferencia frente a scan-to-map y log-odds es menor a medio punto porcentual.

Fase 2: trayectoria y crecimiento de mapa

Escena

Control

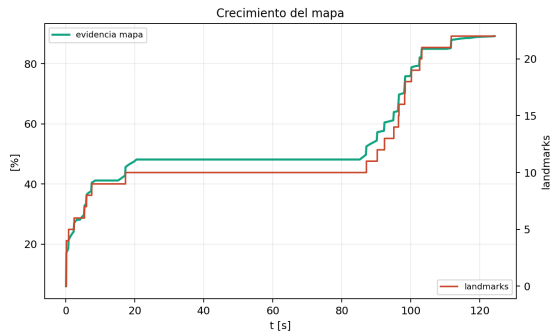
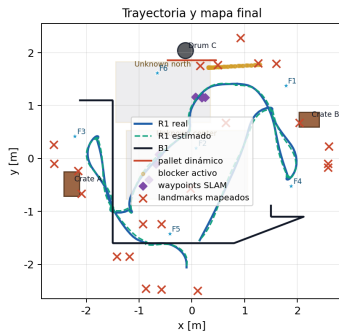
SLAM

Fase 2

Cierre

Trayectoria real vs. estimada · crecimiento de evidencia de mapa durante Fase 2

Phase 2 — trayectoria y crecimiento de mapa



Fase 2: obstáculos y replanning

Escena

Control

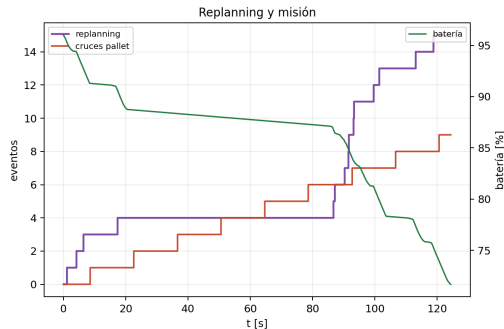
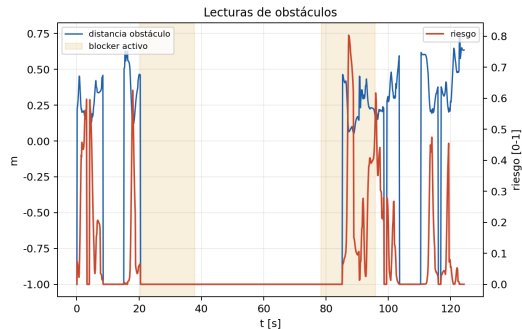
SLAM

Fase 2

Cierre

Distancia a obstáculos, riesgo reactivo, triggers de replanning y batería durante Fase 2

Phase 2 — obstáculos y replanning



Fase 2: señales exportadas

Escena

Control

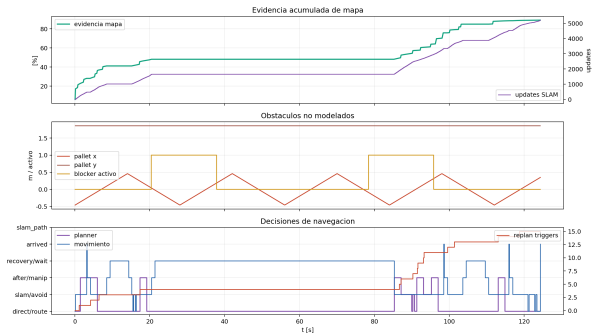
SLAM

Fase 2

Cierre

Las señales reconstruyen la ejecución y comparan navegación, riesgo, mapa y batería.

Series temporales exportadas



Resultado

- Verificar cuándo aparece cada replanificación.
- Medir riesgo máximo y distancia mínima a obstáculos.
- Cruzar mapa revelado con error de pose.
- Comparar consumo de batería entre métodos.

CSV

f fuente de plots y tablas

Comparación Fase 2: métricas

Escena

Control

SLAM

Fase 2

Cierre

Fase 2 se evalúa con cuatro métricas: error, ruta, cobertura y replanificación.

Kalman

88,6 s

menor duración y mapa compacto

Submapas

16 + 4

submapas y cierres de ciclo

Log-odds

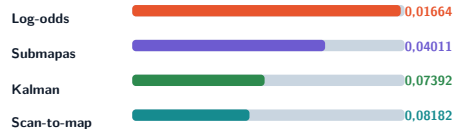
0,017 m

menor RMSE de pose

Robot móvil

4 cruces

obstáculo dinámico evitado

RMSE de pose [m]**Cobertura de mapa [%]**

Elección

Kalman queda primero por error y tiempo; submapas gana en cobertura; scan-to-map acorta ruta; log-odds queda como referencia de grid.

Validación de requisitos

Escena

Control

SLAM

Fase 2

Cierre

Cada punto del enunciado se verifica contra la escena ejecutada y los archivos exportados. Los catorce bloques quedan en estado OK.

- Escena Sim_T2_Phase1_Basic con R1, B1, T1, T2, C1
- Warehouse: mesas, estanterías, sofá y panel de tareas
- Cuatro tareas T1/T2 completadas (entrega y retorno)
- 16 sensores activos y rayos visualizados
- Kalman activo, mapa SLAM actualizado, error acotado
- Batería: tarea aceptada y retorno a carga
- Fase 2: mapa desconocido, robot móvil y 4 SLAM
- Mannequin B1 (Bill) como persona móvil
- Scripts de R1 y B1 adjuntos a la escena
- Bill camina entre estaciones y toma/trabaja/entrega
- Anticolisión: modo AVOIDING y señal de riesgo
- Planificación con SLAM_WAYPOINT
- Capturas 3D desde CoppeliaSim (Fase 1 y Fase 2)
- Deck editable .pptx + PDF exportado

4/4

tareas completadas

16

sensores activos + rayos

0,254 m

error de pose máx. (acotado)

64,7 %

batería al volver a carga

Incidencias corregidas y robustez

Escena

Control

SLAM

Fase 2

Cierre

Cuatro incidencias detectadas en validación y las salvaguardas añadidas para que la misión repita el ciclo sin bloquearse, sin chocar de forma destructiva ni salir de la celda.

Sincronización T2

R1 podía iniciar T2 antes de que Bill llegara a WS2; se bloquea hasta que B1 está en su estación

Ciclos del planificador

celdas de grid persistentes hacían oscilar el desvío; se exige persistencia y se ignoran celdas pegadas

Robot móvil empuja a R1

el robot errante sacaba a R1 de la celda; ahora R1 lo evita con margen, el robot cede el paso y la geo-valla en Z evita la caída

Oscilación Hector

con poca textura el scan-matching daba puntajes planos; ventana pequeña, umbral y ganancia parcial

Salvaguardas de navegación añadidas (Fase 2 multi-ciclo)

Giro 180° de escape

Geo-valla X/Y/Z

El robot cede el paso

Cotas por update del EKF

Throttle de replan

4

incidencias corregidas

0

recuperaciones en 3 ejecuciones

3

vueltas completas sin bloqueo

ni cae

geo-valla X/Y/Z mantiene a R1 dentro

Conclusiones

Escena

Control

SLAM

Fase 2

Cierre

La misión se completa; la elección depende de la métrica priorizada.

Control elegido

PID queda primero en el puntaje de la celda: menor riesgo, menor error de pose y menor duración. NMPC y LQR le siguen; PI queda al final por mayor riesgo.

SLAM elegido

Kalman-landmark da la menor duración y un mapa compacto; GMapping el menor RMSE. Las grids sostienen navegación; los submapas registran estructura con más coste.

Limitaciones explícitas

SLAM simplificado en Lua, rayos de proximidad en lugar de LiDAR real y una ejecución por método en las tablas principales.

Trabajo futuro

LiDAR 2D real, optimización de grafo completa para submapas, repetición estadística y más obstáculos dinámicos.

Cierre

Reúne especificación, implementación Lua, métricas y límites.

Referencias

Escena

Control

SLAM

Fase 2

Cierre

- Siegwart, R.; Nourbakhsh, I.; Scaramuzza, D. (2011). *Introduction to Autonomous Mobile Robots*. MIT Press.
- Tzafestas, S. G. (2014). *Introduction to Mobile Robot Control*. Elsevier.
- LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press.
- Kalman, R. E. (1960). A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering*.
- Thrun, S.; Burgard, W.; Fox, D. (2005). *Probabilistic Robotics*. MIT Press.
- Hooke, R.; Jeeves, T. A. (1961). "Direct Search" Solution of Numerical and Statistical Problems. *Journal of the ACM*.
- Grisetti, G.; Stachniss, C.; Burgard, W. (2007). Improved Techniques for Grid Mapping With Rao-Blackwellized Particle Filters.
- Kohlbrecher, S.; von Stryk, O.; Meyer, J.; Klingauf, U. (2011). Hector SLAM. IEEE SSRR.
- Hess, W.; Kohler, D.; Rapp, H.; Andor, D. (2016). Real-Time Loop Closure in 2D LIDAR SLAM. IEEE ICRA.
- Khatib, O. (1986). Real-Time Obstacle Avoidance for Manipulators and Mobile Robots. *IJRR*.
- CoppeliaSim User Manual; Pioneer 3 Operations Manual; enunciado VIU de la Actividad 2.
- Repositorios consultados/adaptados: OpenSLAM GMapping, hector_slam, Cartographer y PythonRobotics. Asistencia de IA generativa (Google Gemini y Claude) para ideación visual y consulta documental. Código, métricas, validación y conclusiones son del autor; véase `references.bib` para fuentes primarias.